Operating Systems
Chapter 4
Threads

Instructor

Dr.Eng. Mahmoud M. Ismail

Contact: dr.eng.mahmoudmmi@gmail.com
dr.mahmoudmmi@yahoo.com

Thread Idea

A human body has different parts having different functionalities which are working parallelly (Eg: *Eyes*, *ears*, *hands*, etc).

Similarly a single process might have multiple functionalities running parallelly where each functionality can be considered as a thread.

Analogy that illustrates the idea of threads
Let's look at the following simple math problem:
If we ask any student to solve this problem, he will usually solve it in three steps:

5+6+7+8

1) 5 + 6 the result is 11.
2) Adding 7 to 11, so the result is 18.
3) Adding 8 to 18, so the final value is 26.

One-thread process (sequentially)

- Steps are done behind each other (in succession).
- Assume that each step requires a second from student, then he will complete the work in 3 seconds (5 + 6 = 11, 11 + 7 = 18, 18 + 8 = 26).
- Here the steps are taken sequentially, and the student cannot take two steps at the same time, and this is similar to **the normal process (with one thread).**

Multi-threaded operation (parallelism)

- Now, if student brings two of his colleagues to help him solve this problem, the first student can calculate 5 + 6, and the second student calculates 7 + 8, simultaneously (the two steps in one second), and the third student adds the two results in one second
- Here, the problem was solved in two seconds instead of three seconds (i.e., 33% faster than the first).
- The first two steps were done simultaneously, but the third step can only be done after the completion of the previous two steps because it depends on their results.

Here we conclude:

- Doing more than one job at the same time means a process that has more than one thread.
- The threads cannot work in parallel if they depend on each other.

✓ The threads must be independent of each other in order to work together at one time, otherwise we will not benefit from having more than one thread in the process.

Why thread???

- **Process is single stream of execution (statement by statement)**

  E.g. Scanf() // Running → Blocked

➢ **GUI is multiple inputs active ate the same time**

- Consider what happens when a user clicks a button that results in the performance of a time-consuming operation.
- **A single-threaded application would be unresponsive to user until the operation completed.**

✓ If the time-consuming operation is performed in **a separate thread**, **the application remains responsive to the user**.

So, it is required that Single stream of execution → Multiple streams of execution

Multiple streams of execution

Fork() create **another process** at **a cost of memory**

✓ **The solution is to change unit of run time from process to thread**

✓ **The cost of creating thread within a process is less than creating a new process**

Why thread???

```
Class C{
int S;      // attribute - instance variable- global (shared)
            //T1   // T2    //T3   (S is reserved only one time in memory)
}
C c1 =new C();
```

- The cost of creating **threads** within a Class **is less** than creating a new processes **because** S is **shared** among **threads**.
- This requires that both **programming languages** and **OS** must **support** the idea of **threads**

The concept of a process

The concept of **a process** **as embodying** **two characteristics**:

1) Resource ownership:

- A process is allocated **a virtual address space** to hold **the process image** (program, data, stack, and attributes defined in the process control block).
- From time to time, **a process** may be **allocated control** or **ownership** of **resources**, such as main memory, I/O channels, I/O devices, and files.

2) Scheduling/execution:

- A process has **an execution state** (Running, Ready, etc.) and a dispatching priority, and is the entity that is **scheduled** and **dispatched** by the **OS**.
- The process execution follows an execution path (trace) through one or more programs that may be **interleaved** with other processes.

These **two characteristics** are treated **independently** by **OS**

- Unit of **dispatching** is referred to as **a *thread*** or ***lightweight process***
- Unit of **resource of ownership** is referred to as **a process** or **task**

Thread

A Thread is a line of execution that is **part of a process** (a sequential flow of tasks within a **process**).

✓ **Thus, each process must have at least one thread**

Single-threaded process

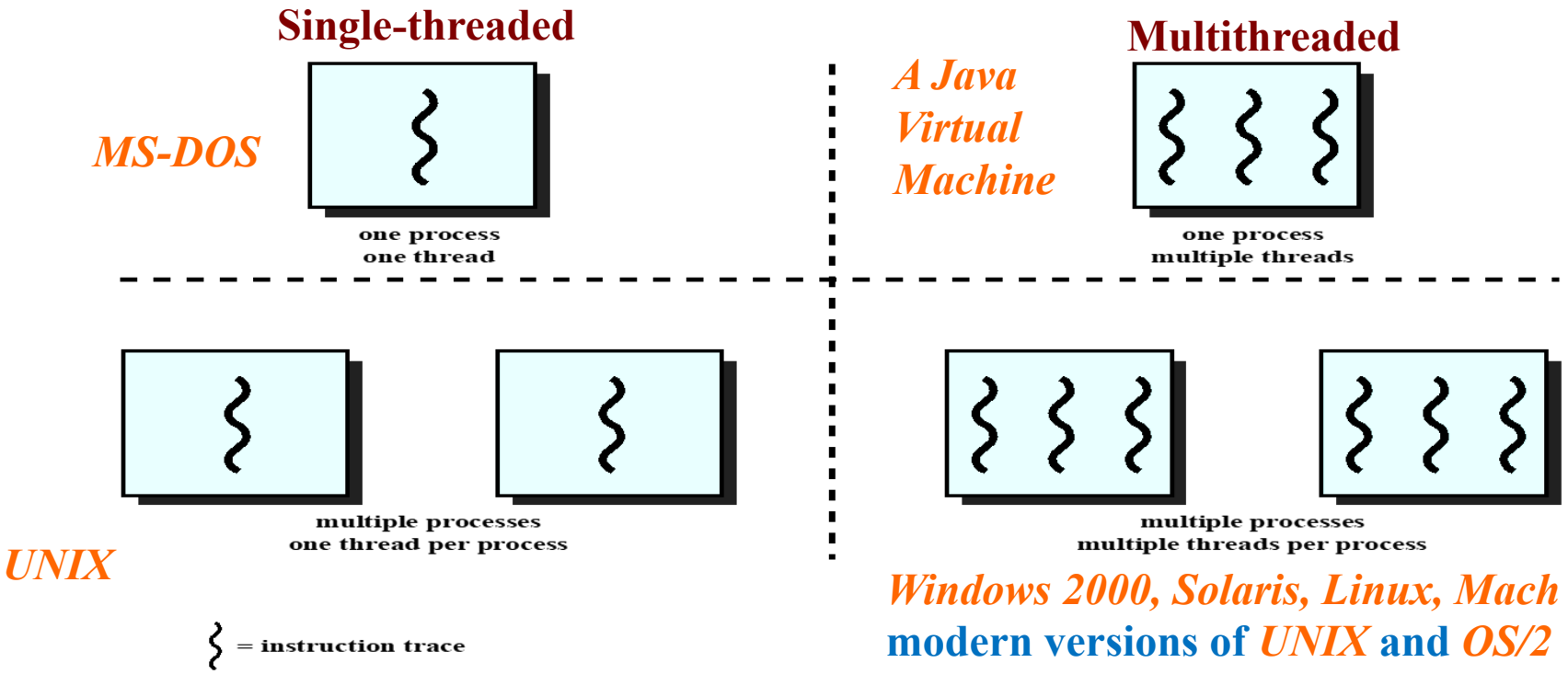
➢ **The process in its normal state performs a single action, and we call it a single thread process.**

➢ **In single-threaded process there is a single point of execution within a program (i.e., a single PC where instructions are being fetched from and executed).**

Multithreading (Multithreaded Process)

- If we want **the process** to perform more than one action at the same time, we must make **the process multi-threaded**.
- **Process** is divided into **threads** that can **run simultaneously**
  - **OS supports multiple threads of execution within a single process by running** different instructions **from the same application.**
  - **Thread is uniquely identified by a Thread ID.**
  - **Perform** more than one task **at the same time** (**concurrent paths of execution within a single process**)

- **In a multi-threaded process there is more than one point of execution/ Multiple streams of execution (i.e., multiple PCs, each of which is being fetched and executed from).**
- **There is a stack per thread.**



The left half of Figure are single-threaded approaches:

- ***MS-DOS*** **supports a single user process and a single thread.**
- **Some traditional *UNIX* systems are multiprogramming systems, thus support multiple user processes but only one thread is allowed for each process.**

The right half of Figure depicts multithreaded approaches:

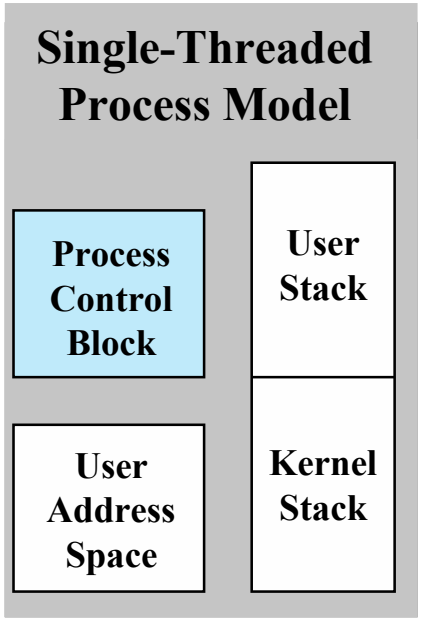
- ***A Java Virtual Machine*** **(JVM) is an example of one process with multiple threads.**
- ***Windows 2000***, ***Solaris***, ***Linux***, ***Mach*** **and many modern versions of *UNIX* and *OS/2*, support multiple processes each of which supports multiple threads.**

Process in a multithreaded environment

- In a multithreaded environment, **a process** is defined as the unit of resource allocation and a unit of protection.
- Associated with **processes**
  - A virtual address space which holds the **process image**
  - Protected access to processors, other processes (for interprocess communication), files, and I/O resources (devices and channels)
- Within **a process**, there may be **one or more threads**

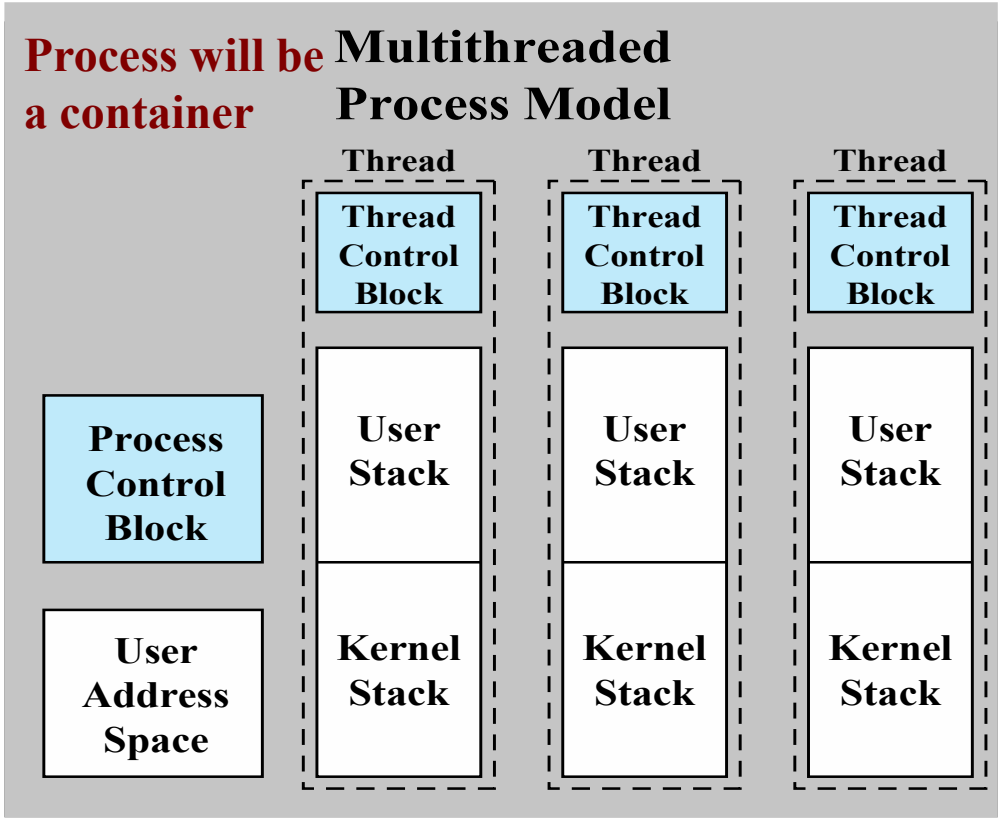
Threads of Process

- Within **a process**, there may be **one or more threads**, each with the following:
  1) **An execution state** (running, ready, etc.)
  2) **Saved thread context** when not running
     – One way to view **a thread** is as an independent program counter operating within **a process**.
  3) Has an execution **stack**
  4) Some **per-thread static storage** for **local variables**
  5) **Access to the memory and resources of its process shared with all other threads in that process**
     – When one thread alters **an item of data** in memory, other threads can **see the result** and **access this item**.



Process includes:

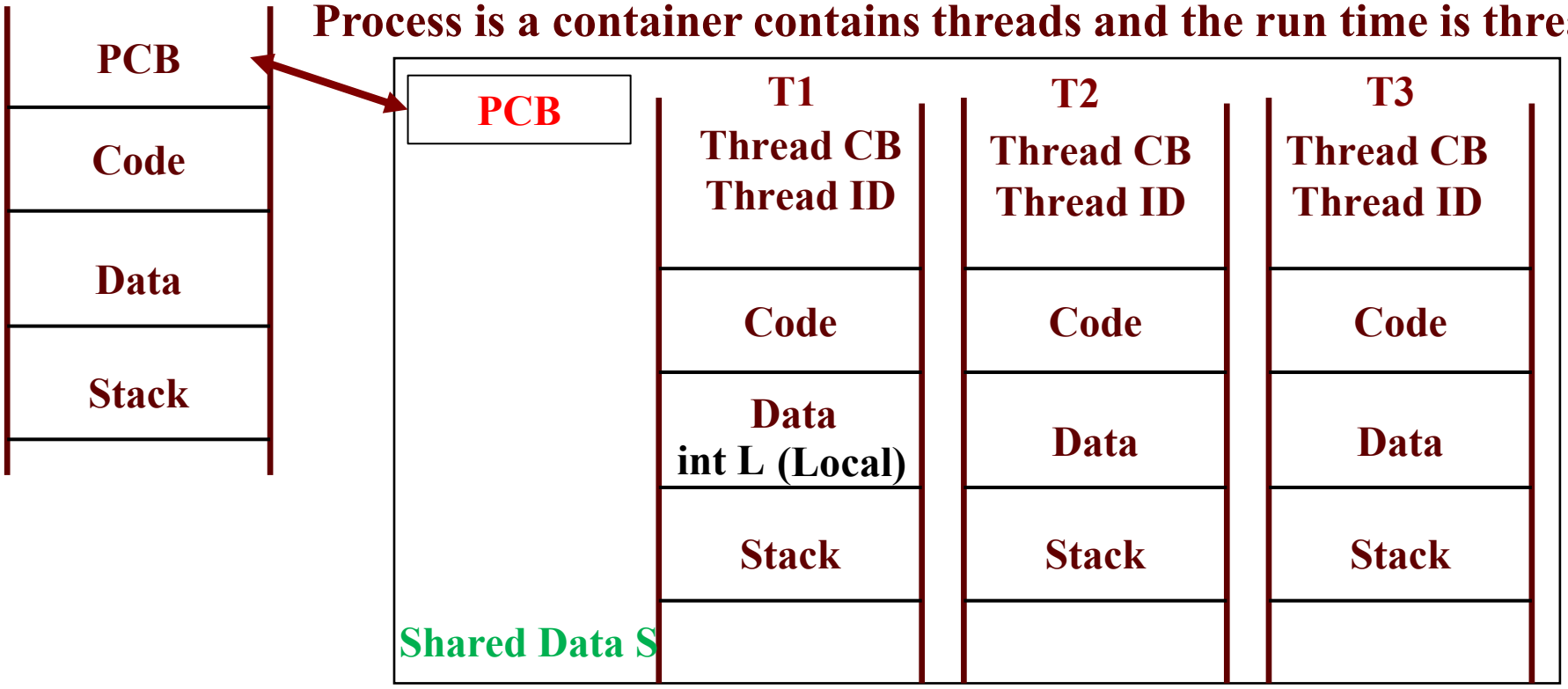
- **Process control block**
- **User address space**,
- **User and kernel stacks** to manage the call/return behavior of the execution of the process.



- There is still a single **process control block**
- **User address space** associated with **the process**, but now there are:
- **Separate stacks** for **each thread**
- **Separate control block** for **each thread** containing **register values**, **priority**, and other **thread-related state information**.

Single-Threaded and Multithreaded Process Models

Process Image




```
Class C{
int S;      //attribute - instance variable- global (Shared Data)
//T1   // T2    //T3   (S is reserved only one time in memory)
T1 extends Threads
{
   int L;  // Local
}
}
```

If we create T4
Allocate PCB ?
Allocate space for shared data S ?

Two programs P1 and P2 with infinite loop
P1 using fork() → Processes
P2 threads
Which one run out of memory faster ???

- **Resource ownership**: **Process** or **threads** ?
  If **T1** open **file** which is the **owner** → **Process**
- **Unit of run time** is **thread**

If we write a program with main **without threads**

✓ *Compiler automatically will create* ***thread*** **because OS** is **Multi-threading OS** (**Unit of run time** by scheduler is **thread** not **processes**)

Benefits of Threads

1) Takes **less time** to **create** a new thread than a process
   - Thread creation is ten times faster than process creation in UNIX
   - In Solaris, creating a process is about thirty times slower than is creating a thread

2) Less time to **terminate** a thread than a process

3) Less time to **switch** between two threads within the same process than to **switch** between processes.

4) Since threads within the same process **share** memory and files, they can **communicate with each other without invoking** the kernel.
   - Communication between independent processes requires the intervention of the kernel to provide protection and the mechanisms needed for communication.

5) It is **faster** to use threads and shared memory than processes and message passing.
   - By default, threads share the memory and the resources of the process to which they belong (Threads use the same data, code and file).
   - Code sharing allows an application to have several different threads of activity all within the same address space.

6) Increases the responsiveness of the user interactive applications

- Program responsiveness enables a program to run even if part of the program is blocked or executing a lengthy operation.
- This quality is especially useful in **designing user interfaces**.

7) Utilization of multiprocessor architectures (increases the concurrency of tasks)

- Multiple threads within the same process can be executing **simultaneously** on different processors.

8) Enhanced throughput of the system (execute multiple tasks of an application at a time).

- Divide a process into multiple threads, and the function of each thread is considered as one job, then the number of jobs completed per unit of time increases which then leads to an increase in the throughput of the system.

Disadvantages of Thread

1) **Resource sharing:**

- Whereas resource sharing is one of the major advantages of threads, it is also a disadvantage because proper synchronization is needed between threads for accessing the shared resources (e.g., data and files).
- Sharing the common data across the threads might cause the data inconsistency or thread sync issues.

➢ **Without proper synchronization** a program where the order of execution is important can cause unexpected results with multiple threads. This is because multiple threads share the same memory allocated to the process.

int S = 50;
S++; // shared data is advantage and disadvantage at the same time

Advantage: all threads aware with update (indirect communication between threads)
Direct communication is disadvantage because it is not scalable (time consuming to inform/connect each thread(my be 1000 threads) with the updates.

Disadvantage: when all threads update S at the same time → data inconsistency
Without management (except there is an order) (Ch.5)

```
Class C{
int S = 10;

        T1
        {
        S++;
        }
        T2
        {
        S++;
        }

void main(void){
  print S;}
}
```

Output ???

If main first executes print S

print → 12

```
void main(void){
Call T1
Call T2
print S;}
```

//No control on order and based on timing (according to which thread the scheduler **selects** and the quantum)

Output ??? 10 or 11 or 12

1) If print **S** is executed first before T1 and T2 → 10
2) If print **S** is executed after T1 or T2 → 11
3) If print **S** is executed after T1 and T2 → 12

- **Code without controlling Timing (no order)**
- **The problem related to shared data when there are W and R (if all R → no problem)**

S++ → **Update S** (W) Print **S** → (R)

This Code allows R and W on **S** **without** any control

Is print first?

Which thread increments first T1 or T2?

Is print before or after T1 increment

Is print before or after T2 increment

1) If all threads **R** no problem
2) If one thread W which is the order R then W or W then R

So, If there is no order **when** R || W→ **inconsistent Data**

The problem is R parallel with W without any control → 10 or 11 or 12

Disadvantages of Thread

2) **Difficult programming model:**
It is difficult to write, debug, test and maintain multithreaded programs for an average user. This is particularly true when it comes to writing code for synchronized access to shared resources.

- Access to **shared data** is an advantage and disadvantage **because** it allows **Concurrent access** to **shared data**.

✓ **Solution: Parallel access→ Serial access How ?**

- Access to **shared data** requires both **Lock** and **Unlock**

  1) Lock data first If succeeded, no others threads can access **shared data** (**no update**)

  2) Unlock data to allow others threads access **shared data** (**Ch.5**)

✓ **Code will also control the order**

- If **required T1** access **before T2** or vice versa
- **In this case**, the code **will force the required order** and will be **independent on timing** of **T1**and **T2** (**independent of which one will get the processor first**)

Key Difference Between Process and Thread

Process	Thread
A Process simply means any program in execution.	Thread simply means a segment of a process.
The process consumes more resources	Thread consumes fewer resources.
The process requires more time for creation, termination and context switching.	Less time for creation, termination and context switching than process.
The process is a heavyweight process	Thread is known as a lightweight process
Has its own Process Control Block, Stack and Address Space	Has Parents' PCB, its own Thread Control Block and Stack and common Address space.
It does not share data (Processes have independent data and code segments)	Threads share data with each other (A thread mainly shares the data segment, code segment, files, etc. with its peer threads)
Communication between processes needs more time as compared to thread.	Communication between threads needs less time as compared to processes.
For some reason, if a process gets blocked then the remaining processes can continue their execution	In case if a user-level thread gets blocked, all of its peer threads also get blocked.
Process is isolated (independent of each other)	Dependent on each other within a process.
E.g.: Running multiple independent applications. Opening two different browsers.	E.g.: Opening two tabs in the same browser.

Scheduling and dispatching is done on a thread basis

- Suspending a process involves suspending all threads of the process since all threads share the same address space
- Termination of a process, terminates all threads within the process

Examples of Multithreading Applications

- Most software applications that run on modern computers are multithreaded.
- An application typically is implemented as a separate process with several threads of control.
- All modern programs that we deal with are made up of a set of threads.
  - ✓ The text editor, browser and all programs are made up of a number of threads. That's because most apps have too many things going on at once.

A word processor may have

1) A thread for responding to keystrokes from the user,
2) Another to perform autosave and
3) A third thread for performing spelling and grammar checking in the background.

- If the text editor program works with one thread, then when the automatic save works, everything must stop until the save is completed.

Examples of Multithreading Applications

A web browser may

1) **Have Multiple tabs**
2) **Have one thread display images or text while another thread retrieves data from the network.**
3) **Able to simultaneously download multiple files from the internet.**
   - **Each file is downloaded in a separate thread, allowing the browser to continue to be responsive to user input while the downloads are taking place.**

- In certain situations, a single application may be required to perform several similar tasks.

For example, a web server accepts client requests for web pages, images, sound, and so forth.

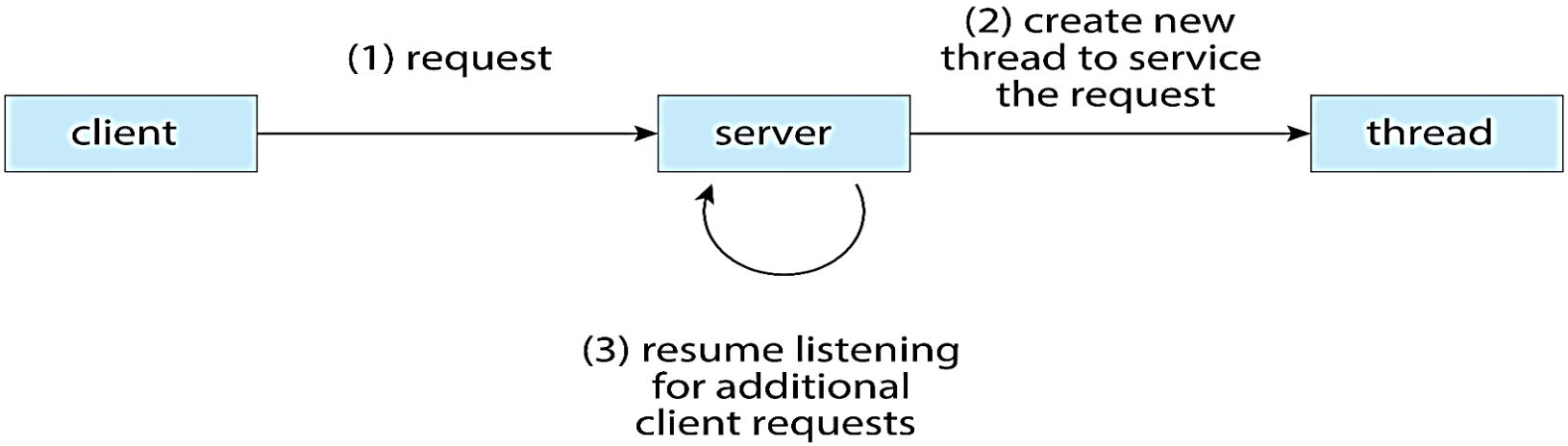
- A busy web server may have several (perhaps thousands of) clients concurrently accessing it.
- If the web server ran as a traditional single-threaded process, it would be able to service only one client at a time, and a client might have to wait a very long time for its request to be serviced.
- If one hundred clients connect to this server at the same time, there will be one thread to serve these clients, one after the other (in succession), so if we assume that each request takes one minute, then the hundredth client will be served after an hour and forty minutes.

- If the new process will perform the same tasks as the existing process, why incur all that overhead?
- It is generally more efficient to use one process that contains multiple threads.

- **Web-server**

➢ Will create a separate thread that listens for client requests.

➢ When a request is made, rather than creating another process, the server creates a new thread to service the request and resumes listening for additional requests.



- A DB Server and network apps that listens for and processes numerous client requests.

➢ With multiple threads running within the same process, switching back and forth among threads invokes less processor overhead compared to major process switching.

- In **a file server** as each new file request comes in, a new thread can be spawned for the file management program.
- Because a server will handle many requests, many threads will be created and destroyed in a short period.
- If the server runs on a multiprocessor computer, then multiple threads within the same process can be executing simultaneously on different processors.
- Further, because processes or threads in a file server must share file data and therefore coordinate their actions, it is faster to use threads and shared memory than processes and message passing for this coordination.
- The thread construct is also useful on a single processor to simplify the structure of a program that is logically doing several different functions.

- ✓ **Most operating-system kernels are now multithreaded.**
- **Several threads operate in the kernel, and each thread performs a specific task, such as managing devices, managing memory, or interrupt handling.**
- ➢ **For example:**
- **Solaris has a set of threads in the kernel specifically for interrupt handling.**
- **Linux uses a kernel thread for managing the amount of free memory in the system.**

Uses of Threads in a Single- User Multiprocessing System

- Foreground to background work
  - One thread displays **menus** and reads user input, while another thread executes user commands and updates spreadsheet.
  - Allowing the program to prompt for the next command before the previous command is complete.
- Asynchronous processing
  - Periodic backup (a word processor to write its RAM buffer to disk once every minute)
- Speed execution
  - Compute one batch of data **while** reading next batch from a device
  - On a multiprocessor system, multiple threads from the same process may be able to execute simultaneously.
- Modular program structure
  - Programs that involve **a variety of activities** may be easier to design and implement using threads.

Thread States

- **Key states**: **Running**, **Ready**, and **Blocked** **Why not suspend?**
- Four basic operations associated with a change in **thread state**

1) Spawn

– Spawn another thread

2) Block

– When a thread needs to wait for an event, it will block (saving its user registers, program counter, and stack pointers).

– The processor may now turn to the execution of another ready thread in the same or a different process.

3) Unblock

– When the event for which a thread is blocked occurs, the thread is moved to the Ready queue.

4) Finish

– Deallocate register context and stacks

Does the blocking of a thread result in the blocking of the entire process? if one thread in a process is blocked, does this prevent the running of any other thread in the same process even if that other thread is in a ready state?

Thread States

- **Key states**: **Running**, **Ready**, and **Blocked**

Why not suspend?

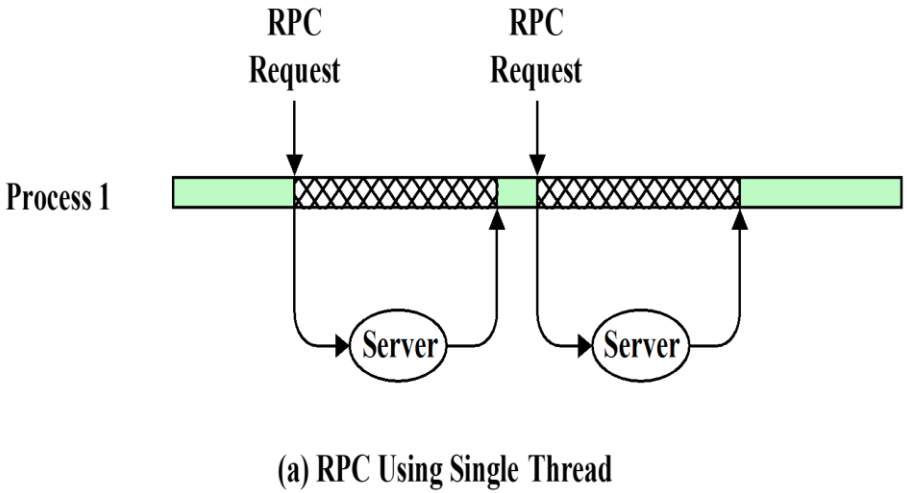
- **Suppose P1 with T1, T2 and T3**
- **Is it logic to suspend T1 (move from memory to secondary storage) ???!!!**
  - **Space** is **shared**/**common** for **all threads**
  - **Threads share the same data**

So, Suspend is a process state not threads state

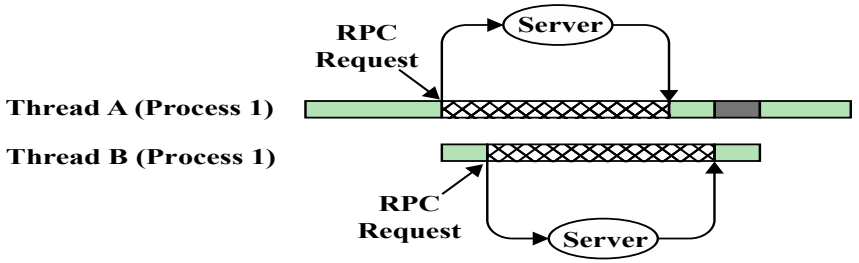
✓ **What is suspended is the all process (all threads) as they share/participate physical space**

Remote Procedure Call (RPC) Using Threads

The performance benefits of threads do not block an entire process



(a) RPC Using Single Thread



(b) RPC Using One Thread per Server (on a uniprocessor)

Blocked, waiting for response to RPC

Blocked, waiting for processor, which is in use by Thread B

Running

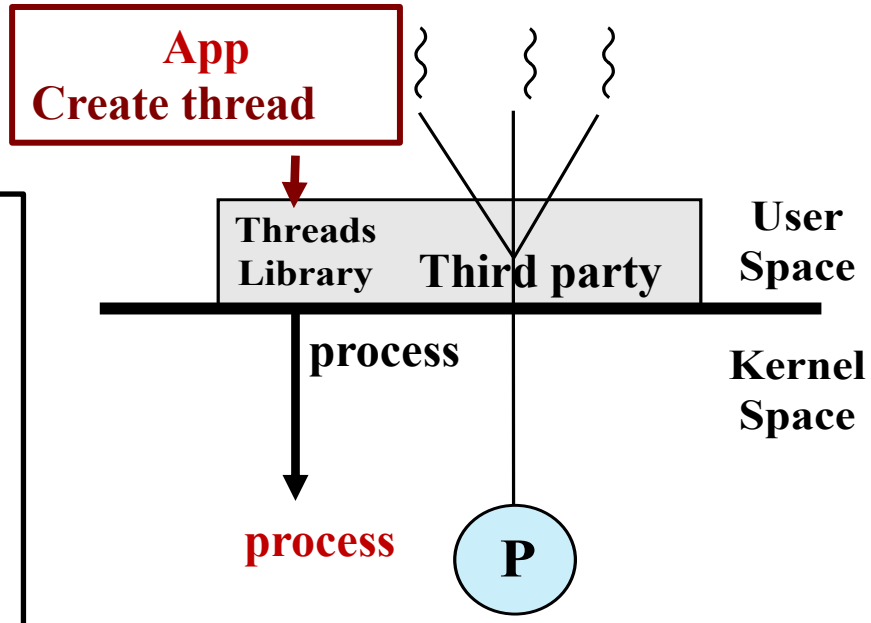
- Program performs two remote procedure calls (RPCs) to **two different hosts** to **obtain a combined result**.
- In a single-threaded program, the results are obtained in sequence, so the program has to wait for a response from each server in turn.
- Rewriting the program to use a separate thread for each RPC results in **a substantial speedup**.

- RPC is a technique by which two programs, which may execute on different machines, interact using procedure call/return syntax.
- Both the called and calling programs behave as if the partner program were running on the same machine. RPCs are often used for client/server applications

How integrate design of threads inside design of OS (Types of Threads)

1) User-Level Threads (ULT)

- The kernel is *not aware* of the existence of threads (Kernel deals with process)
- The creation of the thread and dealing with it takes place at the user level.
- All thread management is done **by the application** through **a thread library** (Pthread/POSIX threads, Java threads) **which deals with** the kernel
- **Thread library** creates a data structure for the new thread and passes control to one of the threads within the process that is in the Ready state using scheduling algorithm.



Pthreads library is implemented at LINUX, UNIX, Solaris, Mac OSX.
POSIX stands for **P**ortable **O**perating **S**ystem **I**nterface [for Unix]

- **Pthread program** must have **pthread.h** header file
- **pthread_create function** creates **thread**
- Implemented either at the ***user*** or ***kernel*** *space*.

Is the creation of new thread leads to new process in kernel?

User-Level Threads (ULT) Advantages

1) Thread switching does not require kernel mode privileges

- Because all of the thread management are within the user address space
- Therefore, the process does not switch to the kernel mode to do thread management.
- This **saves** the overhead of two mode switches (user to kernel; kernel back to user).

2) Can run on any OS

- No changes are required to the underlying kernel to support ULTs.
- The threads library is a set of application-level functions shared by all applications.

3) Scheduling can be application specific

- One application may benefit most from a simple round-robin scheduling algorithm, while another might benefit from a priority-based scheduling algorithm.
- The scheduling algorithm can be tailored to the application **without disturbing** the underlying OS scheduler.

User-Level Threads (ULT) Disadvantages

1) When a thread is blocked, all threads within the process are blocked (the process at the level of kernel will be blocked) → Multiple streams of execution is not achieved

2) Cannot take advantage of multiprocessing

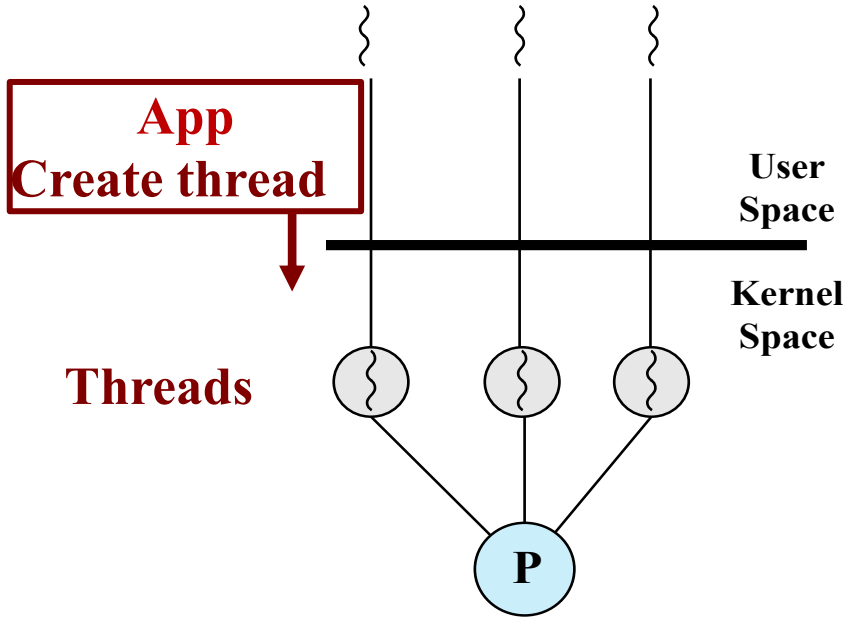
- In a multi-processor system, the kernel assigns one process to only one processor at a time.
- Thus, the multiple threads in a single process have no chance to run simultaneously.

2) Kernel-Level Threads (KLT)

- **Thread management is done by the kernel (OS)**
- **There is no thread management code in the application level**
- **OS provides API to the user application**
- **Scheduling is done on a thread basis**

- An example of an OS that supports kernel threads is the **Windows OS** which has a library that supports threads is the **win32 library**.
- Where you can use **windows.h**, then create the thread with **CreateThread()**.

➢ Linux, and OS/2 are also examples of this approach



Kernel-Level Threads (KLT) Advantages and Disadvantages

Advantages: Overcomes drawbacks of ULT

1) If one **thread** in a process is **blocked**, **the kernel can schedule** another **thread** of **the same process**.
2) The kernel can simultaneously **schedule multiple threads** from **the same process** on **multiple processors**.
3) Kernel routines themselves can be multithreaded.

Disadvantages:

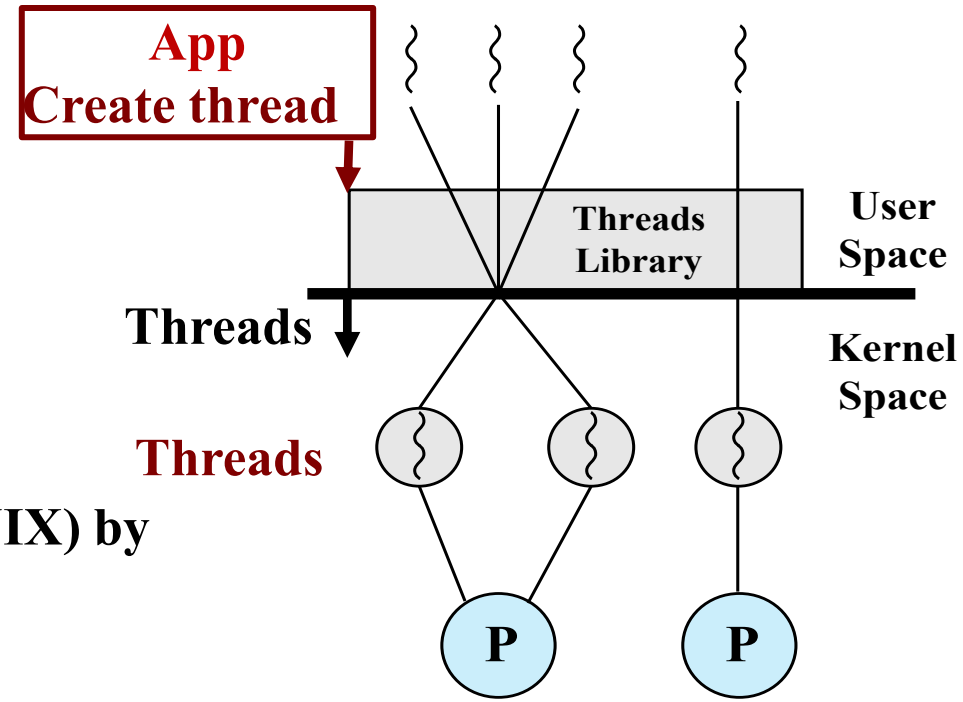
Transfer of control from one thread to another within the same process requires **a mode switch to the kernel** → **overhead**.

KLT is the dominant approach in most OS

User Level threads	Kernel Level Threads
Threads are implemented by users.	These threads are implemented by OS
Threads are not recognized by OS	These threads are recognized by OS
Threads are mainly designed as dependent threads.	These threads are mainly designed as independent threads.
If one user-level thread performs a blocking operation then the entire process will be blocked.	If one kernel thread performs a blocking operation then another thread can continue the execution.
Example of User Level threads: Java thread, POSIX threads.	Example of Kernel level threads: Window, Solaris.
Implementation of User Level thread is done by a thread library and is easy.	While the Implementation of the kernel-level thread is done by OS and is complex.
This thread is generic in nature and can run on any operating system.	This is specific to the operating system.

3) Hybrid Combined Approach (Take the advantages of ULT and KLT)

- **Threads library used to allow app to create threads and managed by it** in the user level
- **Threads library passes threads to kernel**
- Multiple ULTs from a single application are **mapped onto** some (smaller or equal) number of KLTs
- Scheduling and synchronization of threads done in the user level



Example is **Solaris (variant of UNIX) by SUN (Oracle Now)**

Relationship Between Processes and Threads

1) **One-to-One** (Process can create only one thread)

2) **One-to-Many** (Process can create Multiple threads up to physical limit of machine (RAM)

3) **Many-to-One**

4) **Many-to-Many** **(Combines attributes of 1 and 2 cases)**

Many-to-One (Process → thread)

- A thread may migrate from one process environment to another.
- This allows a thread to be **easily moved** among **distinct systems**.
- The movement of that thread among computers may be dictated by the OS for a variety of system-related reasons, such as the need to access **a remote resource**, and **load balancing**.

❖ **T1** belongs to P1

❖ After certain time **T1** belongs to **P2** (Feature called **code mobility**)

➢ OS **provides** a control/management to transfer a thread from one process to another process

❖What about **shared data** ????

e.g. Webserver is an example of network server

Webserver

Client (Browser)



Connect 1 → Thread 1

Connect 2 → Thread 2

Connect N → Thread N



another process

To connect server **(Socket Programming)**

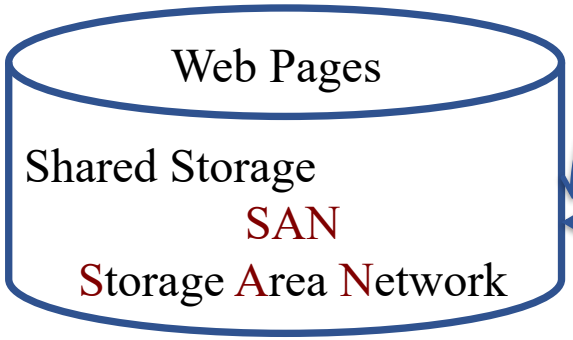
Load → Run out of memory



Webserver

Cluster of Server

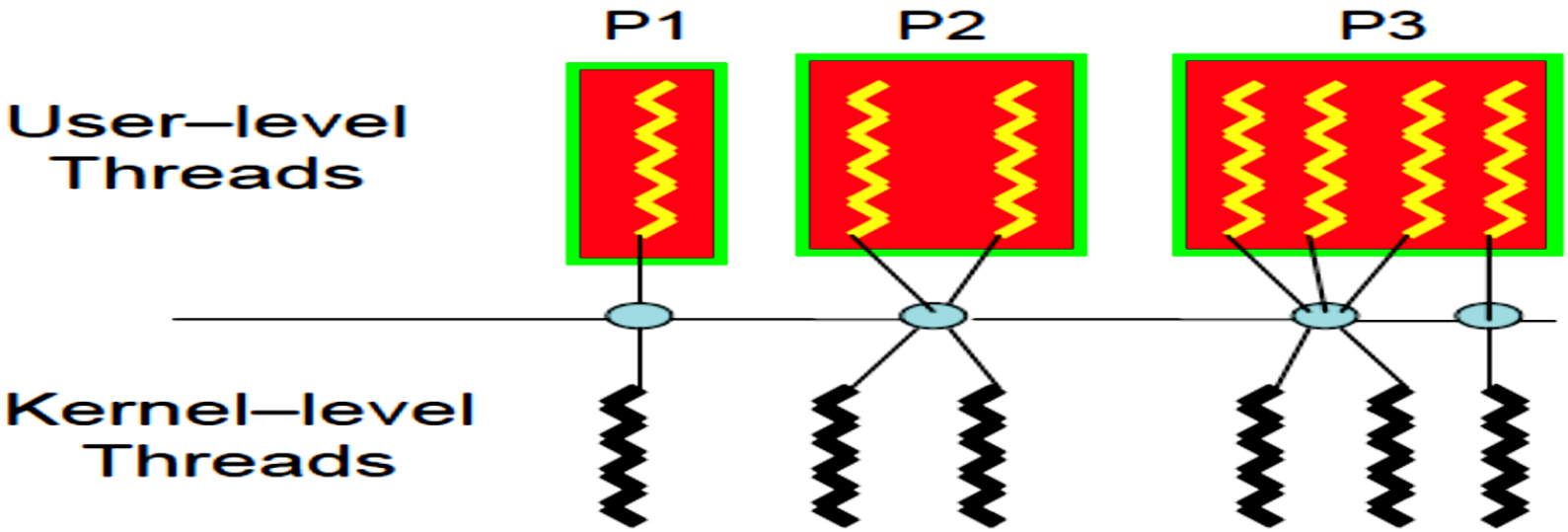
Load Balancing



Used for **Cloud Computing** where **physical machines** contain **virtual machines** and required to move **virtual machine** from **physical machine 1** to **physical machine 2** (transparent to user)

Many-to-Many

- Allows many user level threads to be mapped to a smaller or equal number of kernel threads.
- Blocking the kernel system calls does not block the entire process.
  - When a thread performs a blocking system call, the kernel can schedule another thread for execution.



Relationship Between Processes and Threads

Threads: Processes	Description	Example Systems
1:1	Each thread of execution is a unique process with its own address space and resources.	Traditional UNIX implementations
M:1	Multiple threads may be created and executed within that process.	Windows NT, Solaris, Linux, OS/2, OS/390, MACH
1:M	• A thread may migrate from one process environment to another. This allows a thread to be easily moved among distinct systems. • The movement of that thread among computers may be dictated by the OS for a variety of system-related reasons, such as the need to access a remote resource, and load balancing.	Clouds operating system, and especially its kernel, known as Ra (Clouds), Emerald
M:N	It combines attributes of M:1 and 1:M cases.	TRIX (experimental)


Any Questions ?
Thank you